Lab 12 Answers

Yanqiao Chen
Department of Computer Science and Engineering
Southern University of Science and Technology
12412115@mail.sustech.edu.cn

May 23, 2026

1 Question 1

$$\phi = (x \vee \bar{y} \vee \bar{z}) \wedge (\bar{x} \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z}).$$

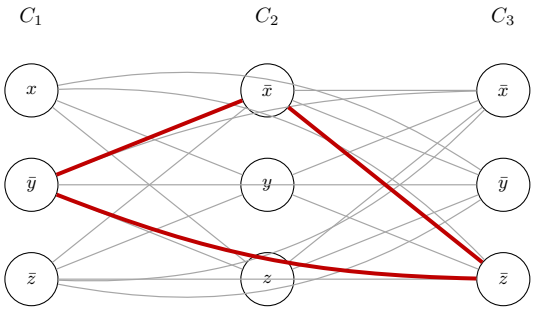
Solution. For the reduction from $3SAT$ to $3CLIQUE$, create one vertex for each literal occurrence in each clause. Join two vertices by an edge if and only if they come from different clauses and their literals are not contradictory. Thus the target clique size is

$$k = 3.$$

Let

$$C_1 = (x \vee \bar{y} \vee \bar{z}), \qquad C_2 = (\bar{x} \vee y \vee z), \qquad C_3 = (\bar{x} \vee \bar{y} \vee \bar{z}).$$

The corresponding graph is:



The red triangle is one 3-clique:

$$\{\bar{y} \in C_1,\ \bar{x} \in C_2,\ \bar{z} \in C_3\}.$$

Since these three literals are mutually consistent, they correspond to a satisfying choice for the three clauses. □

2 Question 2

Solution. Let the input $2SAT$ formula be

$$\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m,$$

where each clause has two literals:

$$C_i = (\ell_{i,1} \vee \ell_{i,2}).$$

We construct a graph G as follows.

- For each literal occurrence $\ell_{i,j}$ in clause C_i, create one vertex $v_{i,j}$.
- Add an edge between $v_{i,j}$ and $v_{p,q}$ if $i \neq p$ and the two corresponding literals are not contradictory. That is, we do not connect x with $\bar{x}$.
- Set the clique size to be
$$k = m.$$

We now prove correctness.

If ϕ is satisfiable, then every clause has at least one true literal. Choose one true literal from each clause. These m chosen literals are from different clauses, and no two of them can be contradictory, since they are all true under the same assignment. Therefore, the corresponding m vertices are pairwise adjacent, so they form a clique of size m.

Conversely, suppose G has a clique of size m. Since vertices from the same clause are not connected, the clique can contain at most one vertex from each clause. There are m clauses and the clique has size m, so it contains exactly one vertex from each clause. Also, since every pair of vertices in the clique is connected, the corresponding literals are mutually consistent. Thus we can assign variables so that all these chosen literals are true. Then each clause has at least one true literal, so ϕ is satisfiable.

The construction creates $2m$ vertices and at most $O(m^2)$ edges, so it runs in polynomial time. Hence

$$2SAT \leq_p CLIQUE.$$

After obtaining this correct reduction, we can conclude that every $2SAT$ instance can be transformed into an equivalent $CLIQUE$ instance in polynomial time. However, since $2SAT \in P$, this reduction alone does not prove that $CLIQUE$ is NP-hard. To prove $CLIQUE$ is NP-hard, we need a reduction from an NP-complete problem such as $3SAT$. □

3 Question 3

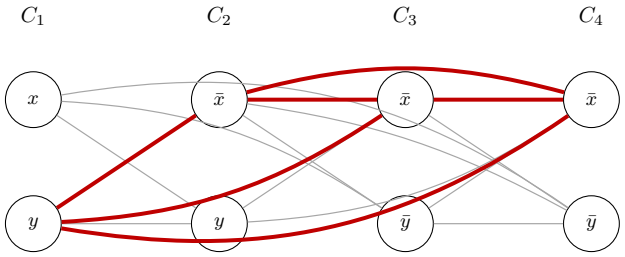
$$\phi = (x \vee y) \wedge (\bar{x} \vee y) \wedge (\bar{x} \vee \bar{y}) \wedge (\bar{x} \vee \bar{y}).$$

Solution. Let

$$C_1 = (x \vee y), \quad C_2 = (\bar{x} \vee y), \quad C_3 = (\bar{x} \vee \bar{y}), \quad C_4 = (\bar{x} \vee \bar{y}).$$

For the reduction to $CLIQUE$, create one vertex for each literal occurrence and connect two vertices if they are from different clauses and are not contradictory. Since there are four clauses, the target clique size is

$$k = 4.$$



The red vertices

$$\{y \in C_1,\ \bar{x} \in C_2,\ \bar{x} \in C_3,\ \bar{x} \in C_4\}$$

form a 4-clique. Therefore, the obtained graph contains a $4CLIQUE$. This corresponds to the satisfying assignment, for example,

$$x = \text{false}, \qquad y = \text{true}.$$

The graph also contains a $3CLIQUE$, because any three vertices from the above 4-clique form a 3-clique. For example,

$$\{y \in C_1,\ \bar{x} \in C_2,\ \bar{x} \in C_3\}$$

is a 3-clique. □

4 Question 4

Solution. First, we show that $\text{Double} - SAT$ is in NP. A certificate consists of two different assignments a_1 and a_2. We can check in polynomial time that $a_1 \neq a_2$ and that both assignments satisfy ϕ.

Then we reduce $3SAT$ to $\text{Double} - SAT$. Given a $3SAT$ formula ϕ, construct

$$\phi' = \phi \wedge (w \vee \bar{w}),$$

where w is a new variable that does not appear in ϕ. This construction is clearly polynomial time.

If ϕ is satisfiable, let a be a satisfying assignment for ϕ. Then extending a with $w = \text{true}$ satisfies ϕ', and extending a with $w = \text{false}$ also satisfies ϕ'. Hence ϕ' has at least two satisfying assignments.

Conversely, if ϕ' has at least two satisfying assignments, then ϕ' is satisfiable. Since $(w \vee \bar{w})$ is always true, the restriction of any satisfying assignment of ϕ' to the variables of ϕ satisfies ϕ. Thus ϕ is satisfiable.

Therefore,

$$\phi \in 3SAT \iff \phi' \in \text{Double} - SAT.$$

So $\text{Double} - SAT$ is NP-hard. Since it is also in NP, it is NP-complete.

□

5 Question 5

$$\phi = (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee z).$$

Solution. Let

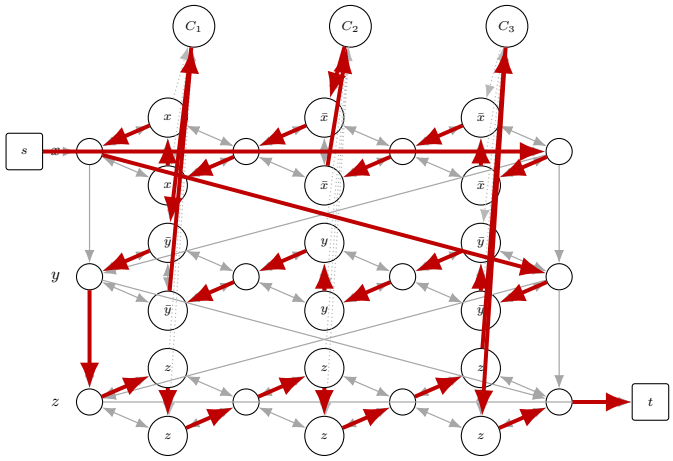
$$C_1 = (x \vee \bar{y} \vee z), \quad C_2 = (\bar{x} \vee y \vee z), \quad C_3 = (\bar{x} \vee \bar{y} \vee z).$$

We construct a directed $HAMPATH$ instance using Sipser-style diamond variable gadgets. Traversing a variable gadget from left to right represents assigning the variable true, and traversing it from right to left represents assigning it false. Clause vertices are connected to the corresponding literal positions in the variable gadgets.

For example, the assignment

$$x = \text{false}, \qquad y = \text{false}, \qquad z = \text{true}$$

satisfies all three clauses. The red path below is the corresponding Hamiltonian path.



The highlighted path visits every vertex exactly once. The path visits C_2 through the literal $\bar{x}$, visits C_1 through the literal $\bar{y}$, and visits C_3 through the literal z. Hence the constructed instance has a Hamiltonian path. □

6 Question 6

Solution. We prove that $3COLOR$ is NP-complete.

First, $3COLOR \in NP$. Given a coloring of the graph, we can check every edge and verify in polynomial time that its two endpoints receive different colors.

Now we reduce $3SAT$ to $3COLOR$. We describe the construction using the following example:

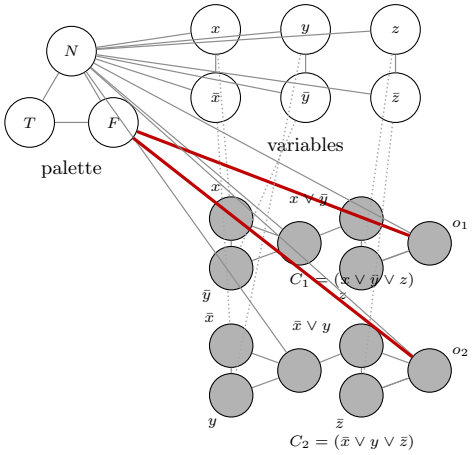
$$\phi = (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee y \vee \bar{z}).$$

The graph uses three special palette vertices T, F, N, forming a triangle. Thus they must receive three different colors. We interpret their colors as True, False, and Neutral.

For each variable, we create two vertices x_i and $\bar{x}_i$, and connect both of them to N and to each other. Therefore $x_i, \bar{x}_i, N$ form a triangle. Since x_i and $\bar{x}_i$ are adjacent to N, neither can receive the neutral color. Since they are also adjacent to each other, one must receive the color of T and the other must receive the color of F.

For a clause $(a \vee b \vee c)$, we use two OR-gadgets: first compute $u = a \vee b$, and then compute $o = u \vee c$. The intermediate output u and the final output o are connected to N, so they must use either the True color or the False color. The final output o is also connected to the palette vertex F. This forces o to be True. Hence the clause gadget is colorable exactly when at least one of a, b, c is colored True.

For the example formula, the construction is illustrated below.



In the figure, each group of gray vertices is a complete OR-gadget. The first triangle computes the OR of the first two literals, and the second triangle combines that value with the third literal. The intermediate and final outputs are connected to N, and the red edges connect the final clause outputs to F.

The key point is the final red edge. If a clause has all three literals colored False, then the OR-gadgets force its output o_i to be colored False. But o_i is adjacent to the palette vertex F, so this is impossible in a proper 3-coloring. On the other hand, if at least one literal in the clause is True, the OR-gadgets allow the output to be colored True, which is compatible with the edges to N and F.

Therefore, the graph is 3-colorable if and only if the original formula is satisfiable. The construction uses a constant-size gadget for each variable and each clause, so it is polynomial time. Since $3SAT$ is NP-complete, this shows that $3COLOR$ is NP-hard. Together with $3COLOR \in NP$, we conclude that $3COLOR$ is NP-complete. $\square$